

Micro-Doppler 集成到 Sionna RT 调研报告

2026年5月12日 | 面向何老师汇报的可行性论证

一、公式层面的区别：标准 Doppler vs Micro-Doppler

1.1 Sionna 当前的标准 Doppler 公式

出自 [paths.py:360–382](#) 文档及 [field_calculator.py:526–562](#) 实现。

Sionna 采用 Wiffen 2018 的多散射点 Doppler 模型（[paths.py](#) 第 367 行引用）。对于一条经历了 n 次散射的路径，Doppler 频移为：

$$f_{\Delta} = (1/\lambda) \cdot [v_0^T k_0 - v_{n+1}^T k_n + \sum_{i=1}^n v_i^T (k_i - k_{i-1})] \quad [\text{Hz}]$$

其中：

- v_0, v_{n+1} = 发射机 / 接收机速度
- v_i ($i = 1 \dots n$) = 场景中第 i 个散射体的速度（来自 `shape` 的 `velocity` 属性）
- k_i = 入射 / 出射波单位方向矢量

每条路径的 Doppler 频移是常数。在 `cir()` 方法中（[paths.py](#) 第 516 行），时间演化为线性相位斜坡：

```
phase = 2π · fΔ · t    # t ∈ [0, 1/W, 2/W, ..., (N-1)/W]
```

即基带等效 CIR 为：

$$a_i^b(t) = a_i \cdot e^{-j2\pi f \tau_i} \cdot e^{j2\pi f_{\Delta} t}$$

1.2 Micro-Doppler 的公式

来源：论文1 "Micro-Doppler Signature Simulation of Multirotor UAVs Using Ray Tracing"（Section II, 第2-3页）及 论文2 "Micro-Doppler Signature-Based Detection, Classification and Localization of Small UAV"（Section 2, 第2-4页）。

旋转叶片的时变径向速度为：

$$v_{\text{blade}}(t) = \omega \times r_{\text{blade}}(t) = 2\pi f_{\text{rot}} \cdot r_{\text{blade}}(t)$$

对应的 micro-Doppler 频移是**时变的**:

$$f_{\text{mD}}(t) = (2 v_{\text{blade}}(t) / \lambda) \cdot \cos(\theta)$$

相位因此多了**正弦调制项**:

$$\phi(t) = 2\pi f_{\Delta} t + \beta \cdot \sin(2\pi f_{\text{rot}} t + \phi_0)$$

其中 **调制指数 β** 为:

$$\beta = (4\pi R / \lambda) \cdot \cos(\theta)$$

- R = 叶片半径
- f_{rot} = 叶片旋转频率
- λ = 载波波长
- θ = 雷达视线与旋转平面的夹角
- ϕ_0 = 叶片初始相位

根据**论文2 第3页 (Equation 7-9)** , 雷达回波可展开为贝塞尔级数:

$$s(t) = \sum A_n \cdot \exp(j2\pi (f_c + f_D + n \cdot f_{\text{rot}}) t)$$

侧带出现在 $n \cdot f_{\text{rot}}$ 的整数倍偏移处, 幅度由贝塞尔函数 $J_n(\beta)$ 加权 (**论文2 Equation 8-9**)。对于多旋翼 UAV, 每根旋翼贡献独立的调制分量 (**论文1 Section II-B, 第3页**)。

1.3 核心差异总结

维度	标准 Doppler	Micro-Doppler
频移特征	常数 f_{Δ}	时变 $f_{\Delta} + f_{\text{mD}}(t)$
相位函数	线性斜坡 $2\pi f_{\Delta} t$	线性斜坡 + 正弦调制 $2\pi f_{\Delta} t + \beta \cdot \sin(2\pi f_{\text{rot}} t + \phi_0)$
频谱特征	单频偏移	以 f_{rot} 为间隔的侧带族
物理来源	刚体平动	旋转 / 振动部件的周期性运动

二、代码层面的定位：Sionna 当前 Doppler 计算流程

2.1 完整流程（按调用顺序）

Step 1 — 射线追踪阶段

文件： `path_solver.py` → `sb_candidate_generator.py` → `image_method.py`

此阶段不涉及任何 Doppler 计算。只确定路径几何（顶点、法线、方向矢量 k_i 等）。

Step 2 — 电磁场传输与 Doppler 计算

文件： `field_calculator.py`, 方法 `_compute_cir()`（第 117–359 行）

Doppler 初始化在第 231 行：

```
doppler = dr.zeros(mi.Float, num_paths) # 初始化 Doppler 为零
```

主循环（第 248 行起）遍历每条路径的每个散射深度，每次迭代调用

`_update_doppler_shift()`（第 300–301 行）：

```
self._update_doppler_shift(shape, doppler, wavelength,
                             ki_world, ko_world, active)
```

Step 3 — `_update_doppler_shift()` 的实现

文件： `field_calculator.py`:526–562

```
v_world = shape.eval_attribute_3("velocity", ...) # 散射体速度
v_effective = dr.dot(ko_world - ki_world, v_world) # 有效速度
doppler_interaction = v_effective / wavelength # Doppler贡献[Hz]
doppler += doppler_interaction # 累加
```

此即公式 $v_i^T(k_i - k_{i-1}) / \lambda$ 的逐项累加实现。

Step 4 — 叠加 TX/RX 端 Doppler

文件： `paths.py`, 方法 `_finalize_doppler_shift_compute()`（第 1215–1246 行）

```
tx_doppler = dr.dot(k_tx, v_tx) / wavelength      # TX 贡献
rx_doppler = dr.dot(k_rx, v_rx) / wavelength      # RX 贡献
doppler = paths_buffer.doppler + tx_doppler - rx_doppler
```

Step 5 — 生成 CIR 的时间演化

文件: `paths.py`, 方法 `cir()` (第 387-524 行)

```
# 基带等效 (第494-499行)
phase = -2π · f · τi
aib = ai · ej·phase

# Doppler 时间演化 (第504-520行) —关键位置!
if num_time_steps > 1:
    time_steps = [0, 1/W, 2/W, ...]
    phase = 2π · doppler · time_steps      # ← 线性相位斜坡
    a = cpx_mul(a, exp)                    # aib(t) = aib · ej2π fΔ t
```

核心发现：整个 Doppler 流程中，`doppler` 张量存储的是**每路径一个标量**（常数频移）。时间演化只在 `cir()` 中通过 $2\pi \cdot \text{doppler} \cdot \text{time_steps}$ 实现。这意味着：只要在 `cir()` 的此处将"线性相位"替换为"线性 + 正弦调制相位"，就完成了 Micro-Doppler 的集成。

2.2 改动点精确标注

改动位置	文件	改动内容	复杂度
主要改动	<code>paths.py</code> 第 516 行 <code>cir()</code> 方法	将 <code>phase = 2π·doppler·t</code> 替换为 <code>phase = 2π·doppler·t + β·sin(2π f_{rot}·t + φ₀)</code>	低
辅助改动 1	<code>paths.py</code> 构造函数 (第 41-145 行)	新增 micro-Doppler 参数张量: <code>_beta</code> , <code>_f_rot</code> , <code>_phi_0</code>	低
辅助改动 2	<code>paths.py</code> <code>_build_from_buffer()</code>	<code>scatter</code> 填充时处理新增张量	低
辅助改动 3	<code>field_calculator.py</code> 或 新增模块	叶片散射模型: 为"叶片路径" 标记并计算 $\beta, f_{\text{rot}}, \phi_0$	中

三、集成方案

3.1 架构决策：不需要修改射线追踪引擎

论文1 Section III（第4-5页）的核心方法论是：射线追踪负责**静态场景**的传播路径计算，旋转叶片的微动效应通过**后处理**在信号级叠加。Sionna 的架构天然支持这种分离：

- 射线追踪层（path_solver / field_calculator）→ 计算静态路径参数 $a_i, \tau_i, f_{\Delta,i}$
- CIR 生成层（paths.py:cir）→ 在时间演化阶段施加相位调制

3.2 具体实现路线

方案 A（最小侵入，推荐）

在 `cir()` 中新增一个可选参数 `micro_doppler_params`，当传入时，将标准 Doppler 相位替换为带调制的相位。

概念性代码（针对 `paths.py` 第 516 行）：

```
# 当前代码
phase = dr.two_pi * doppler * time_steps

# 改后（概念示意）
phase = dr.two_pi * doppler * time_steps # 标准 Doppler 部分
if micro_doppler_params is not None:
    beta, f_rot, phi_0 = micro_doppler_params
    # 调制项:  $\beta \cdot \sin(2\pi f_{\text{rot}} \cdot t + \phi_0)$ 
    modulation = beta * dr.sin(dr.two_pi * f_rot * time_steps + phi_0)
    phase = phase + modulation
```

方案 B（更完整，工作量更大）

在 `Paths` 对象中为每条路径增加一个 `_is_rotor_path` 掩码和对应的 micro-Doppler 参数张量。这样可以对不同路径施加不同的调制参数（如 UAV 有 4 个旋翼，每条叶片反射路径有不同的 β 和 ϕ_0 ），更贴近物理实际（论文1 Section III-B, 第5页）。

3.3 需要新增的参数

参考论文1 Table I（第4页）和论文2 Table I（第5页）的 UAV 参数：

参数	符号	单位	典型值	来源
----	----	----	-----	----

叶片数量	N_{blades}	—	2–4（每旋翼）	论文1 Section III-B
叶片半径	R	m	0.1–0.25	论文2 Table I
旋转频率	f_{rot}	Hz (RPS)	50–150	论文1 Section II-B, 论文2 Table I
观测角	θ	rad	$0-\pi$	论文1 Equation 4
初始相位	ϕ_0	rad	$0-2\pi$ （均匀分布）	论文1 Equation 5

四、可行性结论与工作量估计

4.1 结论

可行。且确实如师兄预期的那样——“只是公式的简单区别，比较好找到位置直接加”。

三个关键判断：

1. **不需要改射线追踪引擎**。改进集中在 `paths.py` 的 `cir()` 方法的相位计算阶段，这是 CIR 生成的后处理环节。
2. **公式差异精确可控**：标准 Doppler 的 $\phi(t) = 2\pi f_{\Delta} t \rightarrow$ Micro-Doppler 的 $\phi(t) = 2\pi f_{\Delta} t + \beta \cdot \sin(2\pi f_{\text{rot}} t + \phi_0)$ ，改动仅需在代码中增加一个正弦项。
3. **Sionna 架构友好**：Paths 类已将路径参数、Doppler 系数和时间演化解耦，可自然扩展现有的 `doppler` 属性为 micro-Doppler 参数。

4.2 需要注意的复杂度

- **物理建模**（论文1 Section III-B, 第4页）：需要定义叶片几何参数和散射点分布。每个叶片可建模为离散散射点集合，每个散射点具有时变位置，其回波贡献随叶片旋转角度变化（闪烁效应）。
- **路径标记**：需要区分常规路径（机身反射）和叶片路径。射线追踪本身不分“静态”和“旋转”对象——需要在追踪后通过形状 ID 或场景标记来区分。
- **时变 RCS**（论文1 Section II-C, 论文2 Section 4）：旋转叶片的 RCS 用叶片闪烁函数（sinc 形式的 blade flash）调制。当前 Sionna 的 a_i 是常数，完整仿真需考虑时变幅值，但这超出了“Micro-Doppler 集成”的核心范畴，可作为后续增强。

4.3 工作量估计

工作项	复杂度	预估工时
在 Paths 类中新增 micro-Doppler 参数字段（ <code>_beta</code> ， <code>_f_rot</code> ， <code>_phi_0</code> ）	低	0.5 天

修改 <code>cir()</code> 方法的相位计算逻辑（第 516 行）	低	0.5 天
编写叶片散射模型（计算 $\beta, f_{\text{rot}}, \phi_0$ 参数）	中	2-3 天
与射线追踪的接口对接（路径标记、场景配置）	中	1-2 天
编写测试用例，对比论文中的 micro-Doppler 频谱结果	中	2-3 天
合计		约 1.5-2 周

4.4 向何老师汇报的核心要点

1. 能做。Micro-Doppler 集成到 Sionna RT 在技术上完全可行。
2. 核心改动位置：`sionna/rt/path_solvers/paths.py` 第 516 行的 `cir()` 方法。
3. 核心公式：在相位中增加 $\beta \cdot \sin(2\pi f_{\text{rot}} t + \phi_0)$ 调制项， $\beta = (4\pi R/\lambda) \cdot \cos(\theta)$ 。
4. 不需要修改射线追踪引擎，不需要动 Mitsuba 渲染内核。
5. 新增参数：叶片半径 R 、旋转频率 f_{rot} 、观测角 θ 、初始相位 ϕ_0 。
6. 工作量：约 1.5-2 周（含物理建模和验证）。

五、参考文献

- [1] "Micro-Doppler Signature Simulation of Multirotor UAVs Using Ray Tracing" — 偏方法，重点章节：**Section II**（Micro-Doppler signal model, 第2-3页）、**Section III**（Ray tracing simulation, 第4-5页）、**Equation 1-8**（调制公式与贝塞尔展开）。
- [2] "Micro-Doppler Signature-Based Detection, Classification and Localization of Small UAV" — 偏应用，重点章节：**Section 2**（Micro-Doppler fundamentals, 第2-4页）、**Section 4**（Feature extraction, 第6-8页）、**Table I**（UAV 参数表）、**Equations 5-9**（雷达回波贝塞尔展开）。

附录：关键文件索引

文件	角色	关键方法
<code>path_solvers/paths.py</code>	Paths 类，存储路径数据，生成 CIR/CFR/taps	<code>cir()</code> , <code>cfr()</code> , <code>taps()</code> , <code>_finalize_doppler_shift_compute()</code>

path_solvers/ field_calculator.py	FieldCalculator, 计算电磁场和延迟	<code>_compute_cir()</code> , <code>_update_doppler_shift()</code> , <code>_update_field()</code>
path_solvers/ paths_buffer.py	PathsBuffer, 路径 缓冲区	<code>add_paths()</code> , <code>shrink()</code>
path_solvers/ path_solver.py	PathSolver, 路径 求解入口	<code>__call__()</code>
rt/scene.py	Scene, 场景管理 (含 velocity 属 性)	场景对象速度接口

— 报告完 —